BestBuy Website Automation Framework

Using Selenium, Pytest & Behave BDD

Capstone Project Report

Submitted By	Arpit Das
Employee ID	4958513
Project Title	BestBuy Website Automation
GitHub Repository	*ArpitDas_4958513_WiproCapstone*

1. Project Title

BestBuy Website Automation Framework

Using Selenium WebDriver, Pytest, and Behave BDD

2. Project Description

2.1 Objective of the Project

The objective of this project is to design and implement a robust, scalable, and maintainable automated testing framework for the BestBuy e-commerce website (www.bestbuy.com). The framework validates critical user journeys — from homepage navigation through product search, filter application, add-to-cart, and checkout — using two complementary testing approaches: Selenium with Pytest (for parametrized data-driven testing) and Behave BDD (for Gherkin-based behaviour-driven testing).

2.2 Overview of Framework

The project is organized into two independent but structurally parallel sub-frameworks housed in the same repository:

- Selenium_BestBuy — A pure Pytest-based test framework implementing the Page Object Model with Excel-driven parametrized tests, Allure reporting, screenshot capture, and structured logging.
- BDD_BestBuy — A Behave BDD framework that maps human-readable Gherkin feature files to Python step definitions, also implementing POM, Excel-driven data, Allure reporting, and an environment.py hooks file.

Both frameworks target the same BestBuy website flows, share the same utility design patterns, and generate Allure reports, making the codebase ideal for demonstrating dual-framework automation expertise.

2.3 Purpose of Automation

Manual testing of an e-commerce website like BestBuy is time-consuming and error-prone, especially for repetitive regression flows such as navigation, product filtering, and cart validation. This framework was created to:

- Reduce manual testing effort and execution time.
- Provide consistent, repeatable test coverage across multiple data combinations.
- Deliver rich, visual Allure reports with screenshots and execution logs for every test step.

- Demonstrate industry-standard framework design patterns (POM, BDD, data-driven testing) as part of a capstone submission.

2.4 Features Automated on the BestBuy Website

The following website features and workflows are covered by the automation framework:

S. No.	Feature	Description
1	Country Selection	Selecting United States on the BestBuy homepage country picker.
2	Top Deals Navigation	Navigating to the Top Deals section from the homepage.
3	Apple Section Navigation	Clicking the Apple brand section from the Top Deals page.
4	MacBook Listing Navigation	Navigating to the MacBook product listing page via href link.
5	Processor Model Filter	Expanding and applying the Processor Model filter panel.
6	Processor Selection (Positive)	Selecting valid processors (Apple M4, Apple M3) using the checkbox filter.
7	Invalid Processor (Negative)	Attempting to select a non-existent processor to validate negative handling.
8	Add to Cart	Adding a filtered MacBook product to the shopping cart.
9	Cart Validation	Verifying that the MacBook product appears correctly in the cart.
10	Checkout Navigation	Proceeding to the Checkout page and verifying the email input field.
11	Invalid Email at Checkout (Negative)	Entering an invalid email on the checkout page and verifying the validation error message.

2.5 Framework Architecture

Both sub-frameworks follow a layered architecture based on the Page Object Model design pattern:

- Configuration Layer — config.properties (Selenium) and config.ini (BDD) store the base URL, browser, and wait settings.
- Page Object Layer — Separate Python classes (HomePage, MacBookPage, CartPage) encapsulate all web element locators and browser interaction methods.
- Data Layer — Excel workbooks (testdata.xlsx) with multiple named sheets supply test inputs to both frameworks.

- Utility Layer — Reusable utility classes for logging (LogGen), screenshots (Screenshot), Excel reading (ExcelUtils), config reading (ReadConfig), and Allure log attachment (AllureLogs — BDD only).
- Test / Step Layer — Pytest test classes (Selenium) and Behave step definition files (BDD) consume the page objects and utilities to execute scenarios.
- Reporting Layer — Allure results are generated automatically after every run and opened in the browser via a post-run hook.

2.6 Technologies Used

Technology	Version / Tool	Purpose
Python	3.x	Primary programming language
Selenium WebDriver	Latest (pip)	Browser automation
Pytest	Latest (pip)	Test runner for Selenium framework
Behave	Latest (pip)	BDD test runner for Gherkin scenarios
ChromeDriver / webdriver-manager	Auto-managed	Chrome browser driver management
Allure Framework	allure-pytest, allure-behave	Rich HTML test reporting
openpyxl	Latest (pip)	Excel data reading (data-driven testing)
pytest-html	Latest (pip)	Additional HTML report (Selenium framework)
configparser	Built-in Python	Reading .ini configuration files (BDD)
Git / GitHub	Latest	Version control and repository hosting

3. Test Strategy

3.1 Type of Framework Implemented

This project implements a Hybrid Automation Framework combining two industry-standard approaches:

- Data-Driven Testing Framework (Selenium + Pytest) — Tests are parametrized using @pytest.mark.parametrize, with all input data sourced from an Excel file. The same test

method runs multiple times with different data combinations (processor types, scenarios, emails).

- Behaviour-Driven Development (BDD) Framework (Behave) — Test scenarios are written in plain English using Gherkin syntax (.feature files), making them readable by non-technical stakeholders. Python step definition files map each Gherkin step to executable automation code.

Both frameworks are built on top of the Page Object Model (POM), ensuring separation of concerns between test logic and UI interaction code.

3.2 Selenium Pytest Implementation

The Selenium_BestBuy framework uses Pytest as the test runner. The key implementation details are:

- conftest.py serves as the Pytest configuration and fixture file. It defines a function-scoped setup() fixture that initializes a Chrome WebDriver with anti-bot detection settings, starts a popup auto-close daemon thread, and tears down the driver after each test via yield.
- pytest.ini configures addopts with --alluredir, --html, and -s -v flags to control test execution and report generation.
- Two test files exist: test_scenarios.py (parametrized scenario tests) and test_end_to_end_flow.py (a single end-to-end flow test).
- All four test classes (TestPositiveFlow, TestNegativeFlow, TestCheckoutNegativeFlow, TestNavigationFlow) are decorated with @allure.feature and @allure.title for Allure categorization.
- A pytest_unconfigure hook in conftest.py automatically runs allure generate and allure open after all tests complete.

3.3 BDD Behave Implementation

The BDD_BestBuy framework uses Behave with the allure-behave formatter. Its implementation includes:

- Two .feature files: end_to_end.feature (single comprehensive end-to-end scenario) and test_scenarios.feature (8 individual scenario test cases — TC_01 through TC_08).
- Two step definition files: end_to_end_steps.py and test_scenarios_steps.py. Each Gherkin step keyword (Given, When, Then, And) maps directly to a Python function decorated with @given(), @when(), or @then() from the behave library.
- behave.ini configures the AllureFormatter (format = allure_behave.formatter:AllureFormatter) and sets the output directory to reports/allure-results.
- environment.py implements the before_scenario(), after_scenario(), and after_all() hooks for driver setup, teardown, failure screenshot capture, and Allure report generation.
- @allure.step decorators are applied to step functions to track each individual step inside the Allure report.

3.4 Page Object Model (POM) Usage

Both frameworks implement POM with three page classes located in their respective pages/ folders:

- HomePage — Encapsulates methods for: open_bestbuy() (loads URL from config), get_title() (returns driver.title), click_country() (selects United States), click_top_deals() (navigates to deals), and click_apple() (navigates to Apple section). All interactions use explicit WebDriverWait with JavaScript clicks and smooth scrollIntoView.
- MacBookPage — Encapsulates methods for: click_macbook() (navigates to MacBook listing), select_processor_filter() (expands filter panel), select_processor(processor) (selects checkbox and returns True/False for positive/negative testing), click_listing_add_to_cart(processor) (clicks the specific product's Add to Cart button by data-testid), and click_go_to_cart() (navigates to cart).
- CartPage — Encapsulates methods for: verify_cart_item() (checks for MacBook in page source), click_checkout() (clicks Checkout button and waits for full DOM load), enter_invalid_email(email) (types invalid email), verify_invalid_email_error() (checks for error text in page source), and click_continue() (submits the checkout form).

Each page class accepts the driver object via its constructor (__init__), stores it as self.driver, and creates a self.wait = WebDriverWait instance for explicit waits.

3.5 Data-Driven Testing Implementation

Data-driven testing is implemented via the ExcelUtils utility class (utilities/excel_utils.py) and an Excel workbook (data/testdata.xlsx) with four named sheets:

Sheet Name	Test Cases	Columns / Data Used
PositiveData	TC_01 to TC_04	TC_ID, Processor (Apple M4 / Apple M3), Scenario (AddToCart / Checkout / ValidateCart)
NegativeData	TC_05, TC_06	TC_ID, Invalid Processor name, Scenario (NegativeFilter)
CheckoutNegativeData	TC_07	TC_ID, Invalid Email address, Scenario (InvalidEmail)
NavigationData	TC_08	TC_ID, Scenario (NavigationValidation)

The ExcelUtils class provides two static methods:

- get_all_data(path, sheet_name) — Reads all rows from row 2 onwards (skipping header) and returns a list of tuples. Used in Selenium Pytest tests with @pytest.mark.parametrize to iterate over all rows.
- get_data_by_tc_id(path, sheet_name, tc_id) — Iterates rows to find the row where column[0] matches the given TC_ID. Used in BDD step definitions to fetch specific test data by scenario ID.

3.6 Logging Implementation

Logging is implemented through the LogGen utility class (utilities/logger.py), which uses Python's built-in logging module:

- LogGen.loggen() is a static factory method that creates and returns a configured logger instance.
- The logger writes to logs/automation.log at INFO level with the format: %(asctime)s - %(levelname)s - %(message)s.
- A duplicate handler check (if not logger.handlers) prevents multiple handlers from being attached when the logger is called multiple times across test iterations.
- In the Selenium framework, pytest.ini also enables live console logging via log_cli = true and log_cli_level = INFO, so all logger.info() calls are visible in the terminal during execution.
- In the BDD framework, behave.ini sets stdout_capture = false and log_capture = false so print statements and logs stream to the console in real-time.

3.7 Screenshot Capturing Mechanism

Screenshots are captured by the Screenshot utility class (utilities/screenshot.py) and are taken at every significant test step in both frameworks:

- Screenshot.capture(driver, screenshot_name) creates the reports/screenshots/ directory if it does not exist.
- Each screenshot is saved with a timestamped filename in the format: {screenshot_name}_{YYYYMMDD_HHMMSS}.png.
- After saving, the screenshot is immediately attached to the Allure report using allure.attach.file() (Selenium) or allure.attach() with driver.get_screenshot_as_png() (BDD), using attachment_type=allure.attachment_type.PNG.
- In the BDD environment.py, an additional screenshot is captured on test failure inside after_scenario() when scenario.status == 'failed', saved as {scenario.name}.png.
- Screenshots are taken at every major step: Homepage, Country Selection, Top Deals, Apple Section, MacBook, Processor Filter, Processor Selected, Add to Cart, Cart Page, Checkout, and Invalid Email — providing a complete visual audit trail.

3.8 Reporting Implementation

Both frameworks use Allure as the primary reporting tool. The reporting pipeline works as follows:

- During test execution, allure-pytest (Selenium) and allure-behave (BDD) collect test results and write JSON result files to reports/allure-results/.
- After all tests finish, a hook automatically runs the Allure CLI command: allure generate reports/allure-results -o reports/allure-report --clean, which produces a full HTML report in reports/allure-report/.
- The allure open reports/allure-report command then opens the report in the default browser automatically.
- For the Selenium framework, pytest_unconfigure(config) in conftest.py serves as the post-run hook. For BDD, after_all(context) in environment.py performs the same task.
- The Selenium framework also generates a pytest-html report (report.html) via the --html flag in pytest.ini.

3.9 Reusability and Scalability Approach

The framework is designed for reusability and scalability through the following design decisions:

- Page Object Model ensures that any change to a UI locator only needs to be updated in one place (the relevant page class), not across multiple test files.
- Utility classes (LogGen, Screenshot, ExcelUtils, ReadConfig) are shared modules that can be reused across any number of test files without duplication.
- The config files (config.properties / config.ini) externalize environment-specific values (base URL, browser, waits), making it easy to switch environments without changing test code.
- The Excel-based data layer allows QA teams to add new test scenarios simply by adding rows to the relevant sheet, without modifying any Python code.
- New page flows can be added by creating a new page class and a new feature file / test class, without disrupting existing tests.

4. Test Environment

4.1 Environment Summary

Component	Details
Operating System	Windows 10 / 11 (compatible with macOS and Linux)
Browser	Google Chrome (latest stable version)
Browser Driver	ChromeDriver — auto-managed via webdriver-manager (ChromeDriverManager().install())
Programming Language	Python 3.x
IDE	PyCharm (Community or Professional)
Test Runner (Selenium)	Pytest
Test Runner (BDD)	Behave
Reporting Tool	Allure Framework (allure-pytest, allure-behave) + pytest-html (Selenium only)
Version Control	Git, GitHub (Repository: ArpitDas_4958513_WiproCapstone)
Application Under Test	https://www.bestbuy.com/

4.2 Libraries and Dependencies

Selenium_BestBuy/requirements.txt

```
selenium
pytest
webdriver-manager
allure-pytest
pytest-html
openpyxl
allure-python-commons
```

BDD_BestBuy/requirements.txt

```
allure-python-commons
selenium
behave
webdriver-manager
allure-behave
pytest
openpyxl
configparser
gherkin-official
```

4.3 Framework Setup Requirements

To set up and run the framework on a local machine, the following steps are required:

1. Install Python 3.x from https://www.python.org/downloads/
2. Install the Allure command-line tool (allure) and ensure it is available in the system PATH.
3. Clone the repository: git clone https://github.com/Ady218/ArpitDas_4958513_WiproCapstone.git
4. Navigate to the desired framework folder (Selenium_BestBuy or BDD_BestBuy).
5. Install dependencies: pip install -r requirements.txt
6. Ensure Google Chrome is installed. ChromeDriver is managed automatically by webdriver-manager.
7. Run the tests using the commands described in Section 5.

5. Test Execution

5.1 How Selenium Pytest Tests Are Executed

The Selenium_BestBuy framework is executed using the pytest command from the Selenium_BestBuy/ directory. The pytest.ini file pre-configures the execution options, so no additional flags are needed beyond the basic command.

5.2 Pytest Execution Commands

Run All Tests

```
cd Selenium_BestBuy
pytest
```

Run Only the End-to-End Test

```
pytest tests/test_end_to_end_flow.py
```

Run Only Scenario Tests

```
pytest tests/test_scenarios.py
```

Run with Verbose Output

```
pytest -v -s
```

Manually Generate and Open Allure Report

```
allure generate reports/allure-results -o reports/allure-report --clean
allure open reports/allure-report
```

> 📌 **The allure generate and allure open commands are called automatically via the pytest_unconfigure hook in conftest.py after every test run. Manual execution is only needed if the hook is skipped.**

5.3 How Behave BDD Tests Are Executed

The BDD_BestBuy framework is executed using the behave command from the BDD_BestBuy/ directory. The behave.ini file configures the AllureFormatter and output directory automatically.

Run All BDD Scenarios

```
cd BDD_BestBuy
behave
```

Run a Specific Feature File

```
behave features/test_scenarios.feature
behave features/end_to_end.feature
```

Run a Specific Scenario by Name

```
behave --name "TC_01 Validate Apple M4 Add To Cart Flow"
```

Run with Console Output

```
behave --no-capture
```

5.4 Feature File Execution Flow

When behave is executed, it follows this flow:

8. Behave reads behave.ini and discovers .feature files inside the features/ directory.
9. For each Feature, it reads the Scenarios defined within.
10. Before each Scenario, the before_scenario() hook in environment.py is called, which initializes the Chrome WebDriver and stores it in context.driver.
11. Each Gherkin step (Given, When, Then, And) is matched against step definitions in the features/steps/ directory using the @given(), @when(), @then() decorators and exact string matching.
12. The matched Python function is executed, performing the browser action via the page object, capturing a screenshot, and logging the result.
13. After each Scenario, after_scenario() is called: if the scenario failed, a failure screenshot is saved; the driver is then closed.
14. After all scenarios complete, after_all() runs the allure generate and allure open commands.

5.5 How Behave Maps Feature Files to Step Definitions

Behave uses string matching between Gherkin step text and Python decorator strings to link steps to their implementations. For example:

Feature file step:

```
Given user loads positive test data "TC_01"
```

Step definition in test_scenarios_steps.py:

```
@given('user loads positive test data "{tc_id}"')
def step_load_positive_data(context, tc_id):
    ...
```

The quoted parameter {tc_id} in the decorator is automatically extracted from the step text and passed as a function argument. Behave scans all Python files inside features/steps/ to find matching decorators, so all step definitions must be in that directory.

5.6 How Data Is Fetched from Excel and Executed

In Selenium Pytest (test_scenarios.py)

At module load time (before any test runs), ExcelUtils.get_all_data() reads all rows from each sheet of data/testdata.xlsx and returns a list of tuples. These lists are passed to @pytest.mark.parametrize, which automatically creates a separate test instance for each data row.

```
test_data = ExcelUtils.get_all_data('data/testdata.xlsx', 'PositiveData')
@pytest.mark.parametrize('tc_id, processor, scenario', test_data)
```

```
def test_positive_flow(self, setup, tc_id, processor, scenario):
```

In BDD Behave (step definitions)

When a Given step such as 'user loads positive test data "TC_01"' is executed, ExcelUtils.get_data_by_tc_id() is called with the specific TC_ID extracted from the Gherkin text. The returned row tuple is unpacked and stored in the context object (context.tc_id, context.processor, context.test_scenario), making the data available to all subsequent steps within the same scenario.

```
data = ExcelUtils.get_data_by_tc_id('data/testdata.xlsx', 'PositiveData', tc_id)
context.tc_id = data[0]
context.processor = data[1]
context.test_scenario = data[2]
```

5.7 Hooks / Environment Execution Flow (BDD)

The environment.py file in BDD_BestBuy/features/ defines three Behave lifecycle hooks:

Hook	What It Does
before_scenario(context, scenario)	Prints scenario start banner. Initializes Chrome WebDriver with anti-bot options (--disable-blink-features=AutomationControlled, excludeSwitches, useAutomationExtension=False). Sets implicit wait (5s). Removes navigator.webdriver property via JS. Starts popup auto-close daemon thread.
after_scenario(context, scenario)	Checks if scenario.status == 'failed'. If failed, saves a screenshot named after the scenario to reports/screenshots/. Calls context.driver.quit() to close the browser. Prints scenario completion banner.
after_all(context)	Runs allure generate reports/allure-results -o reports/allure-report --clean via os.system(). Then runs allure open reports/allure-report to launch the report in the browser.

5.8 Selenium Pytest conftest.py Execution Flow

The conftest.py file manages the Pytest test lifecycle:

Hook / Fixture	What It Does
setup() fixture	Function-scoped Pytest fixture. Configures Chrome with anti-bot detection. Starts popup handler thread. Yields the driver to the test. Calls driver.quit() after the test completes.
auto_close_popup(driver)	A background daemon thread function that waits up to 3 seconds for a 'No, Thanks' popup button and clicks it via JavaScript if found.
pytest_unconfigure(config)	Called after all tests complete. Prints completion message. Runs allure generate and allure open commands to produce and display the final Allure report.

6. Test Output

6.1 Console Logs

During test execution, the following information is streamed to the console:

- Pytest: With log_cli = true and -s -v flags configured in pytest.ini, the console shows each test name, PASSED/FAILED status, all logger.info() messages (prefixed with timestamp and level), and print statements such as POPUP CLOSED.
- Behave: With stdout_capture = false and log_capture = false in behave.ini, the console shows scenario start banners (STARTING: <scenario name>), step-level pass/fail, print statements from step functions, and scenario completion banners (COMPLETED: <scenario name>).

6.2 Automation Log File

Both frameworks write to a dedicated log file at logs/automation.log using Python's logging module. Log entries follow the format:

```
2024-01-15 10:23:45,123 - INFO - Opening BestBuy
2024-01-15 10:23:48,456 - INFO - Selecting Country
2024-01-15 10:23:51,789 - INFO - Selecting Processor: Apple M4
2024-01-15 10:24:02,012 - INFO - Screenshot saved at:
reports/screenshots/TC_01_04_Processor_20240115_102402.png
2024-01-15 10:24:10,345 - INFO - Positive Test Passed
```

The log file provides a complete chronological record of all automation actions, which is valuable for debugging failures and audit trails.

6.3 Screenshot Generation

Screenshots are saved to reports/screenshots/ with the naming convention {TC_ID}_{step_number}_{step_name}_{timestamp}.png. Examples of generated screenshot filenames:

```
TC_01_01_Homepage_20240115_102301.png
TC_01_02_Apple_20240115_102345.png
TC_01_03_MacBook_20240115_102410.png
TC_01_04_Processor_20240115_102502.png
TC_01_05_AddToCart_20240115_102530.png
TC_01_06_Cart_20240115_102555.png
```

For the end-to-end test, screenshots are named with descriptive step labels: 01_Homepage_Opened, 02_Country_Selected, 03_Top_Deals_Page ... 10_Checkout_Page.

6.4 Pass / Fail Validations and Assertions

The framework uses Python assert statements throughout the test code. The following assertions are implemented:

Assertion	Location	Validates
assert 'Best Buy' in home.get_title()	All test flows	Homepage title contains 'Best Buy'
assert 'bestbuy' in driver.current_url.lower()	Country selection	URL contains bestbuy after country selection
assert 'apple' in driver.page_source.lower()	Apple navigation	Apple page content is present
assert 'macbook' in driver.page_source.lower()	MacBook navigation	MacBook listing page loaded
assert processor_found is True	Positive flow	Valid processor checkbox found and clicked
assert processor_found is False	Negative flow	Invalid processor not found (negative validation)
assert cart.verify_cart_item()	Cart validation	'MacBook' present in cart page source
assert email_field.is_displayed()	Checkout validation	Email input field visible on checkout page
assert cart.verify_invalid_email_error()	Invalid email (negative)	'valid email' text present in page source

6.5 Dynamic Execution Flow (Scenario-Driven)

The Positive Flow test in Selenium (TestPositiveFlow) implements a dynamic execution path based on the 'scenario' column read from the PositiveData Excel sheet. Within a single parametrized test method, the execution branches into one of three paths:

- AddToCart — Navigates through the full flow, adds the filtered MacBook to cart, opens the cart, and asserts the URL contains 'cart'.
- Checkout — Performs the same flow as AddToCart, then proceeds to the checkout page and asserts that the email input field is displayed.
- ValidateCart — Performs the AddToCart flow and asserts that cart.verify_cart_item() returns True (MacBook in page source).

This single parametrized method therefore covers multiple test scenarios from a single code block, driven purely by Excel data.

7. Test Reports

7.1 Allure Report Implementation

Allure is the primary reporting framework used in both sub-frameworks. It is integrated as follows:

- Selenium_BestBuy: The allure-pytest plugin is enabled via --alluredir="reports/allure-results" in pytest.ini. @allure.feature(), @allure.title(), and screenshot attachments populate the report.
- BDD_BestBuy: The allure-behave formatter is configured in behave.ini (format = allure_behave.formatter:AllureFormatter, outfiles = reports/allure-results). @allure.step() decorators on step functions and AllureLogs.attach_log() calls enrich the report.
- After execution, both frameworks run: allure generate reports/allure-results -o reports/allure-report --clean, producing a self-contained HTML report.

7.2 Allure Report Contents

The generated Allure report contains the following sections and details:

Overview / Summary Dashboard

- Total test count with pass/fail/broken/skipped breakdown displayed as a donut chart.
- Suites hierarchy showing Features → Test Classes → Individual Tests.
- Test execution timeline showing the start time, duration, and status of each test.

Test Suites

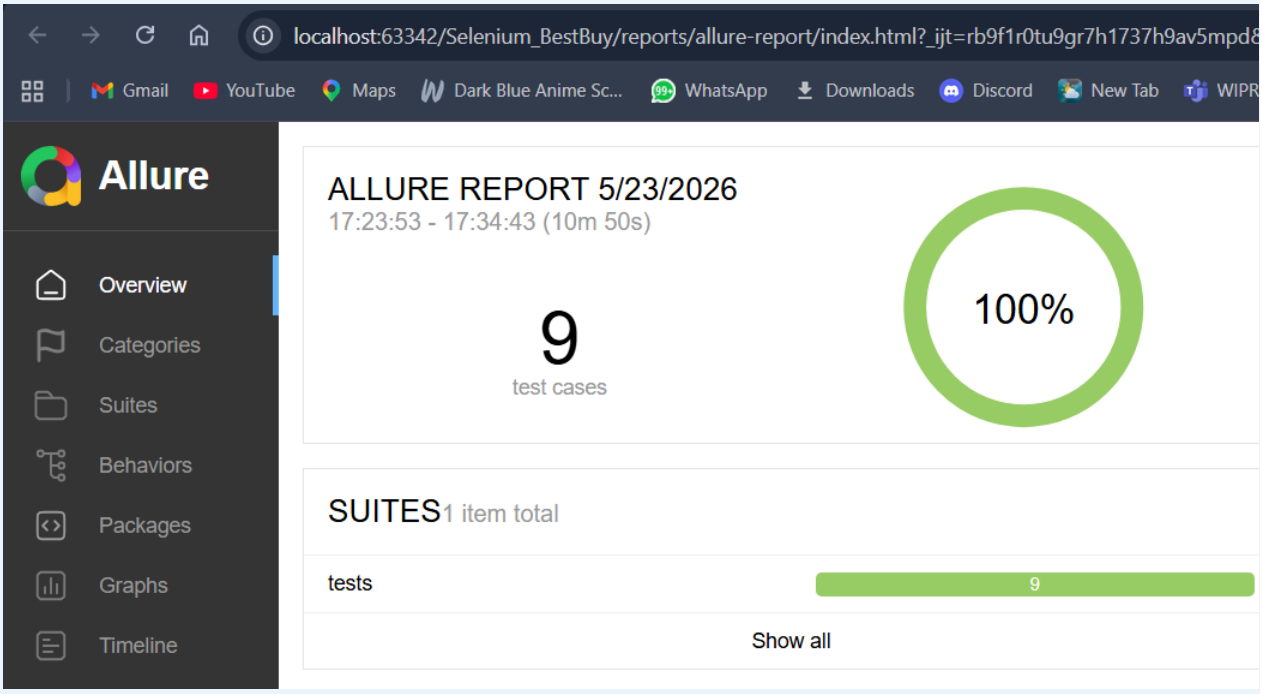
- For Selenium: Organized by @allure.feature() values — 'BestBuy Positive Automation', 'BestBuy Negative Automation', 'Checkout Negative Automation', 'BestBuy Navigation Automation', 'BestBuy End To End Automation'.
- For BDD: Organized by Feature name from the .feature file — 'BestBuy End To End Automation' and 'BestBuy Scenario Validations'.

Individual Test Detail View

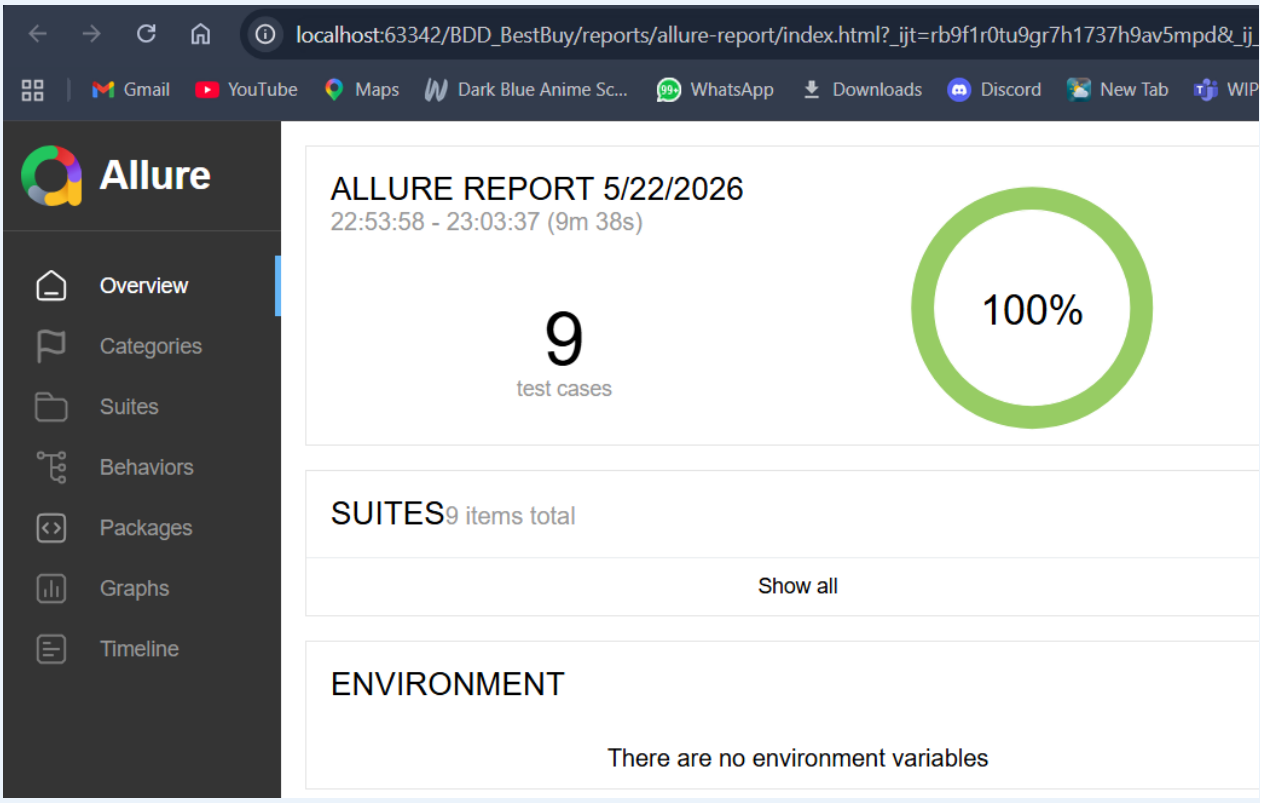
- Test name (@allure.title value or Scenario name from Gherkin).
- Status (Passed / Failed / Broken).
- Duration of each test and each step.
- Parametrize values (tc_id, processor, scenario) for Selenium parametrized tests.
- Step-by-step execution trace with each step name, status, and duration.
- All screenshots attached inline within the relevant step — visible by clicking the attachment in the report.
- Execution Log text attachments (AllureLogs.attach_log() entries) visible as TEXT attachments in BDD steps.

7.3 Screenshot Attachments in Allure

Every call to Screenshot.capture() automatically attaches the screenshot to the current Allure context. In the Selenium framework, allure.attach.file() attaches the saved PNG file by path. In the BDD framework, allure.attach() uses driver.get_screenshot_as_png() to embed the image directly as binary PNG data. The attachment is visible in the Allure report under the corresponding test step, with the screenshot_name as the attachment label.



[SCREENSHOT] — *Allure Report Selenium Dashboard (Overview)*



[SCREENSHOT] *Allure Report BDD Dashboard (Overview)*

7.4 Execution Logs in Reports

Log messages are attached to Allure in two ways:

- Selenium framework: The console log output (enabled by log_cli = true in pytest.ini) is captured by allure-pytest and attached as a 'log' section in each test's detail view.
- BDD framework: Each step function calls AllureLogs.attach_log(message), which uses allure.attach() with attachment_type=allure.attachment_type.TEXT to create a named 'Execution Log' text attachment for every step, providing a step-level log trail inside the report.

7.5 Scenario Hierarchy in Allure

The Allure report presents tests in a three-level hierarchy:

- Feature Level: Defined by @allure.feature() (Selenium) or the Feature: keyword in the .feature file (BDD). Examples: 'BestBuy Positive Automation', 'BestBuy Scenario Validations'.
- Test / Scenario Level: Defined by @allure.title() (Selenium) or the Scenario: keyword in Gherkin (BDD). Examples: 'Positive Ecommerce Flow [TC_01-Apple M4-AddToCart]', 'TC_07 Validate Invalid Checkout Email'.

- Step Level (BDD only): Each @allure.step()-decorated step function appears as a named sub-step in the test timeline, e.g., 'Load Positive Test Data', 'Open BestBuy Website', 'Select Processor Filter'.

7.6 Failure Tracking

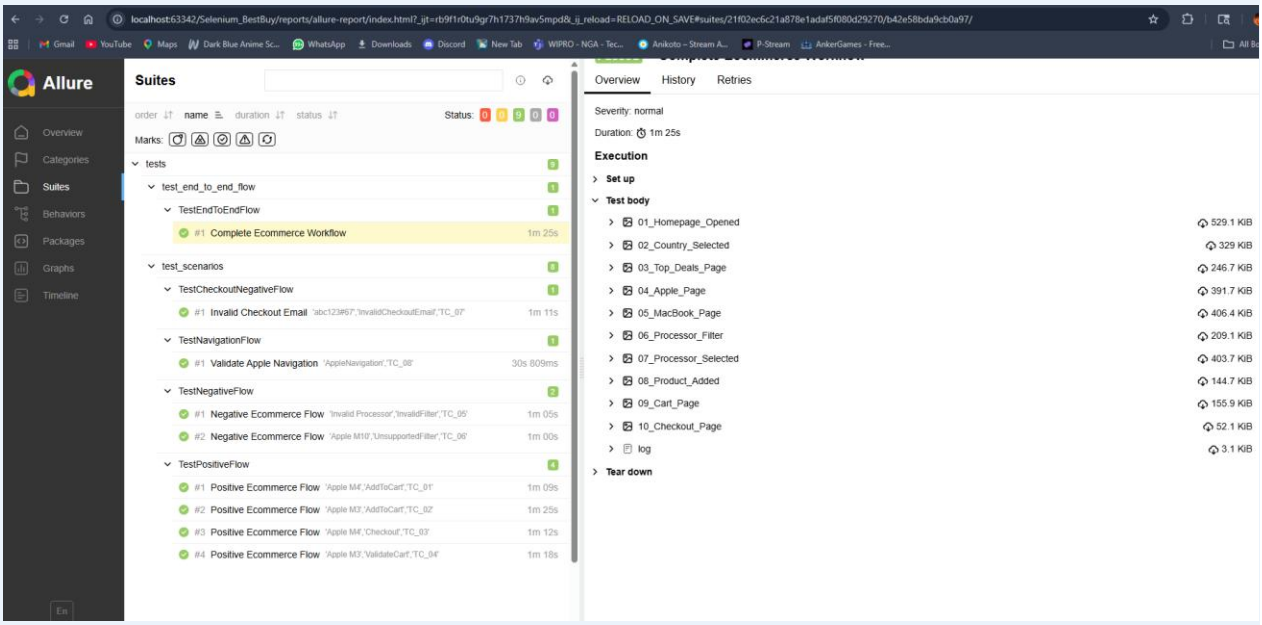
When a test fails:

- Pytest (Selenium): The assertion error message is displayed in the test detail view. The failure screenshot (taken just before the assertion in Screenshot.capture()) is attached and visible in the report.
- Behave (BDD): The failed step is highlighted in the Allure report. The after_scenario() hook captures an additional failure screenshot named after the scenario and saves it to reports/screenshots/. The error traceback is shown in the step detail.
- Allure's Categories and Timeline views allow filtering by failed tests to quickly identify the failing scenarios and their failure points.

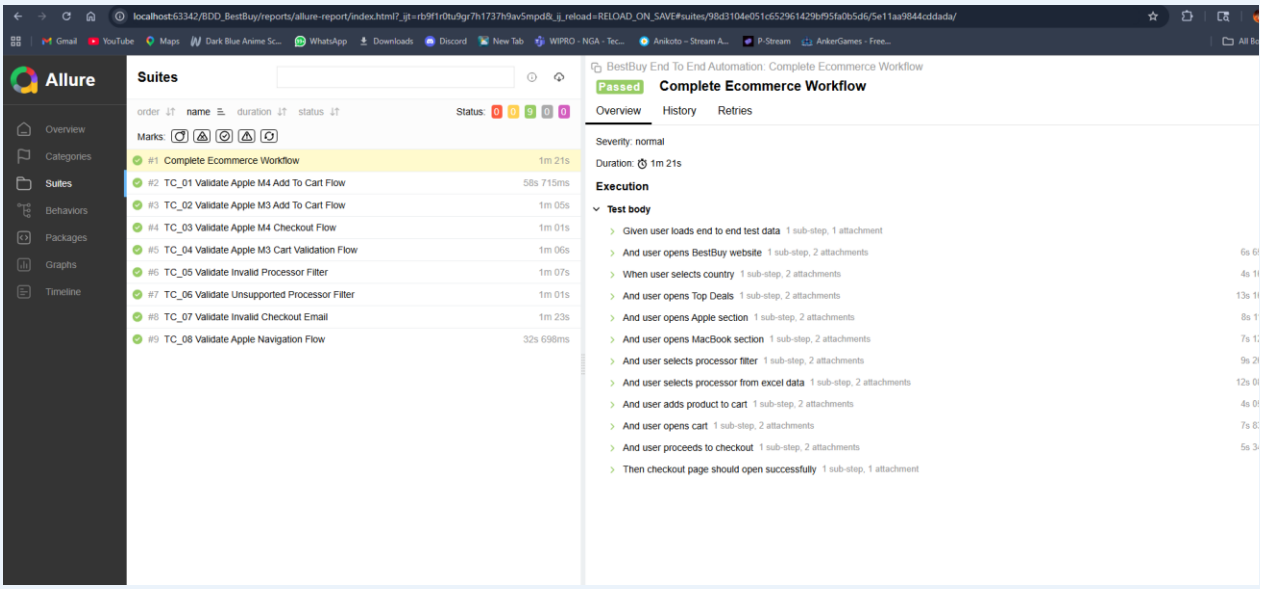
7.7 Report Generation Process Summary

The complete report generation process from execution to viewing:

15. Execute tests: pytest (Selenium) or behave (BDD).
16. During execution: allure-pytest / allure-behave write JSON result files to reports/allure-results/.
17. Post execution: The hook (pytest_unconfigure / after_all) automatically calls allure generate reports/allure-results -o reports/allure-report --clean.
18. The HTML report is built in reports/allure-report/ with all assets (JS, CSS, images) embedded.
19. allure open reports/allure-report launches a local web server and opens the report in the default browser.



[SCREENSHOT] — *Allure Report Selenium Suites Hierarchy*



[SCREENSHOT] — *Allure Report BDD Suites Hierarchy*

8. Project Folder Structure

The complete folder structure of the project repository is as follows:

8.1 Selenium_BestBuy Folder Structure

File / Folder	Description
Selenium_BestBuy/	Root of the Selenium Pytest framework
conftest.py	Pytest fixtures, Chrome driver setup, popup handler, post-run Allure hook
pytest.ini	Pytest configuration — alluredir, html report, log_cli settings
requirements.txt	Python package dependencies
config/config.properties	Base URL, browser, timeout configuration
data/testdata.xlsx	Excel workbook with PositiveData, NegativeData, CheckoutNegativeData, NavigationData sheets
pages/__init__.py	Package initializer
pages/home_page.py	HomePage class — country, top deals, apple navigation
pages/macbook_page.py	MacBookPage class — macbook nav, processor filter, add to cart, go to cart
pages/cart_page.py	CartPage class — verify cart, checkout, invalid email, continue
tests/__init__.py	Package initializer
tests/test_scenarios.py	4 parametrized test classes: Positive, Negative, CheckoutNegative, Navigation
tests/test_end_to_end_flow.py	Single end-to-end test class: TestEndToEndFlow
utilities/__init__.py	Package initializer
utilities/logger.py	LogGen class — Python logging factory
utilities/screenshot.py	Screenshot class — capture, save, and attach to Allure
utilities/excel_utils.py	ExcelUtils class — get_all_data() and read_data() methods
utilities/read_properties.py	ReadConfig class — reads config.properties for URL, browser, waits

logs/automation.log	Auto-generated log file during test execution
reports/screenshots/	Auto-generated step-level PNG screenshots
reports/allure-results/	Raw Allure JSON result files generated by allure-pytest
reports/allure-report/	Final rendered HTML Allure report

```
Selenium_BestBuy/
│
├── config/
│   └── config.properties
│
├── data/
│   └── testdata.xlsx
│
├── logs/
│
├── pages/
│   ├── __init__.py
│   ├── home_page.py
│   ├── macbook_page.py
│   └── cart_page.py
│
├── reports/
│
├── tests/
│   ├── __init__.py
│   ├── test_end_to_end_flow.py
│   └── test_scenarios.py
│
├── utilities/
│   ├── __init__.py
│   ├── excel_utils.py
│   ├── logger.py
│   ├── read_properties.py
│   └── screenshot.py
│
```

```
├── pytest.ini
├── package.json
├── package-lock.json
├── README.md
├── conftest.py
└── requirements.txt
```

8.2 BDD_BestBuy Folder Structure

File / Folder	Description
BDD_BestBuy/	Root of the Behave BDD framework
behave.ini	Behave configuration — AllureFormatter, output directory, capture settings
requirements.txt	Python package dependencies for BDD framework
config/config.ini	Base URL, browser, implicitWait, explicitWait settings
data/testdata.xlsx	Same sheet structure as Selenium — PositiveData, NegativeData, CheckoutNegativeData, NavigationData
features/environment.py	Behave hooks — before_scenario(), after_scenario(), after_all()
features/end_to_end.feature	Gherkin feature file — 1 end-to-end scenario with Excel data loading
features/test_scenarios.feature	Gherkin feature file — 8 scenarios (TC_01 to TC_08) covering all flow types
features/steps/__init__.py	Package initializer
features/steps/end_to_end_steps.py	Step definitions for end_to_end.feature
features/steps/test_scenarios_steps.py	Step definitions for test_scenarios.feature — all TC_01–TC_08 steps
pages/__init__.py	Package initializer
pages/home_page.py	HomePage class (identical design to Selenium version)
pages/macbook_page.py	MacBookPage class (identical design to Selenium version)
pages/cart_page.py	CartPage class (identical design to Selenium version)

utilities/__init__.py	Package initializer
utilities/logger.py	LogGen class — identical implementation to Selenium version
utilities/screenshot.py	Screenshot class — uses allure.attach() with get_screenshot_as_png()
utilities/excel_utils.py	ExcelUtils class — includes get_data_by_tc_id() in addition to get_all_data()
utilities/read_properties.py	ReadConfig class — uses configparser to read config.ini
utilities/allure_logs.py	AllureLogs class — static attach_log() method for TEXT attachments in Allure
logs/automation.log	Auto-generated log file during BDD test execution
reports/screenshots/	Step screenshots + failure screenshots from after_scenario() hook
reports/allure-results/	Raw Allure JSON result files generated by allure-behave formatter
reports/allure-report/	Final rendered HTML Allure report

```
BDD_BestBuy/
│
├── config/
│   └── config.ini
│
├── data/
│   └── testdata.xlsx
│
├── features/
│   │
│   ├── steps/
│   │   ├── __init__.py
│   │   ├── end_to_end_steps.py
│   │   └── test_scenarios_steps.py
│   │
│   ├── end_to_end.feature
│   ├── test_scenarios.feature
│   └── environment.py
```

```
│
├── logs/
│
├── pages/
│   ├── __init__.py
│   ├── home_page.py
│   ├── macbook_page.py
│   └── cart_page.py
│
├── reports/
│
├── utilities/
│   ├── __init__.py
│   ├── allure_logs.py
│   ├── excel_utils.py
│   ├── logger.py
│   ├── read_properties.py
│   └── screenshot.py
│
├── behave.ini
├── README.md
└── requirements.txt
```

— End of Report —

BestBuy Website Automation Framework | Arpit Das | 4958513 | Wipro Talent Next Program